

YouTube Tutorial Speaking Guide

Video goal

Create a clear portfolio/tutorial video that shows how TodoAppWasm was built and deployed as a real full-stack application.

Target length: 25 to 35 minutes.

Main message:

QUOTE: I built this full-stack .NET 8 Todo application with help from AI coding tools, but I reviewed, tested, deployed, debugged, and integrated the application myself.

Recording tools

Recommended setup:

- Recording: OBS Studio
- Editing: Microsoft Clipchamp or DaVinci Resolve
- Upload: YouTube Studio

Use OBS if you want the best free screen recording option. Use Clipchamp if you want simple trimming, titles, and export after recording.

Screen setup

Use two screens:

- Large monitor: record this screen only.
- Laptop screen: keep your notes, ChatGPT/Codex, and checklist there.

Before recording:

- Close private tabs.
- Hide browser bookmarks if they contain personal links.
- Do not show Render database passwords.
- Do not show GitHub secrets.
- Do not show JWT keys.
- Use zoom level 110% or 125% so viewers can read code.
- Prepare test username and password before starting.

Video title ideas

- Deploy a Full-Stack .NET 8 Blazor App to GitHub Pages and Render
- Blazor WebAssembly + ASP.NET Core API + PostgreSQL Deployment Tutorial
- Full-Stack .NET Portfolio Project: Blazor, Render, GitHub Pages, PostgreSQL

Opening script

Hi everyone, my name is Umar.

In this video, I am going to show my full-stack TodoAppWasm project. It is built with Blazor WebAssembly on the frontend, ASP.NET Core Web API on the backend, Entity

Framework Core for data access, JWT authentication, and PostgreSQL for hosted data.

The frontend is deployed to GitHub Pages, the backend is deployed to Render as a Docker image, and the Docker image is published through GitHub Actions to GitHub Container Registry.

I used AI tools as a coding assistant during this project, but I reviewed the code, tested the application, handled the deployment, fixed the errors, and connected the full application end to end.

Section 1: Live demo

Show:

- <https://umarsalem.github.io/ToDoAppWasm/>
- Create user
- Login
- Create todo
- Refresh page
- Show todo still exists

Speaking notes:

This is the live frontend hosted on GitHub Pages. The frontend is a static Blazor WebAssembly application. When I create a user or log in, the browser is calling my live backend API on Render.

After login, the backend returns a JWT token. The Blazor app stores that token in browser local storage and sends it with protected Todo requests.

Section 2: Architecture overview

Show README architecture section.

Speaking notes:

The architecture is simple but realistic.

The Blazor frontend is hosted on GitHub Pages. It calls the ASP.NET Core Web API hosted on Render. The API validates JWT tokens, runs business logic, and stores data using Entity Framework Core in PostgreSQL.

For deployment, GitHub Actions builds and tests the solution, builds the backend Docker image, pushes it to GitHub Container Registry, and the frontend workflow deploys the Blazor app to GitHub Pages.

Section 3: Backend code overview

Show:

- WebAPI/Program.cs
- WebAPI/Auth/JwtTokenService.cs
- WebAPI/Controllers/UsersController.cs
- WebAPI/Controllers/TodosController.cs

Speaking notes:

In Program.cs, the backend registers controllers, Swagger, authentication, authorization, CORS, and Entity Framework Core.

The database provider is configurable. Locally I can use SQLite, but in Render I set DatabaseProvider to Postgres. This lets the same application run locally and in production without changing source code.

The JWT settings come from environment variables. That is important because secrets should not be committed to GitHub.

The health endpoint is also important for hosting:

```
text
/health
```

Render uses this endpoint to check whether the service is alive.

Section 4: Database setup

Show:

- EfcDataAccess
- EfcDataAccess.Postgres
- Render PostgreSQL dashboard without secrets visible

Speaking notes:

For local development, the app can use SQLite. For hosted deployment, I use PostgreSQL because it is better for a real hosted application and data survives redeloys.

This project keeps PostgreSQL migrations separate from SQLite migrations. That matters because database providers can generate different column types and migration code.

In Render, the connection string is added as an environment variable:

```
text
ConnectionStrings__TodoDatabase
```

The double underscore is important in ASP.NET Core configuration.

Section 5: Docker and GitHub Actions

Show:

- WebAPI/Dockerfile
- .github/workflows/development-ci.yml
- GitHub Actions run
- GitHub Container Registry package

Speaking notes:

The backend is deployed as a Docker image. The Dockerfile builds and publishes the WebAPI project, then starts the app inside a Linux container.

The GitHub Actions workflow runs on the development branch. It restores packages, builds the solution, runs tests, builds the Docker image, and pushes the image to GitHub Container Registry.

The image tag used by Render is:

text
ghcr.io/umarsalem/todoappwasm:development

Section 6: Render backend deployment

Show Render service settings, but hide secrets.

Show:

- Image URL
- Health check path
- Environment variable names only
- Deploy logs after successful deploy
- /health endpoint
- /swagger endpoint

Speaking notes:

In Render, I created a Web Service from an existing Docker image. I used the GHCR image built by GitHub Actions.

The important environment variables are:

text
DatabaseProvider=Postgres
ApplyMigrationsOnStartup=true
ConnectionStrings__TodoDatabase=<secret>
AllowedOrigins=https://umarsalem.github.io
Jwt__Key=<secret>
Jwt__Issuer=TodoAppWasm.WebAPI
Jwt__Audience=TodoAppWasm.BlazorApp
Jwt__TokenLifetimeMinutes=60

I also set the health check path to:

text
/health

After deployment, I tested the backend using:

text

<https://todoappwasm-api.onrender.com/health>

<https://todoappwasm-api.onrender.com/swagger>

Section 7: Frontend deployment

Show:

- .github/workflows/blazor-github-pages.yml
- GitHub secret API_BASE_URL
- GitHub Pages deployment
- Live frontend

Speaking notes:

The frontend is deployed to GitHub Pages. The workflow publishes the Blazor WebAssembly app and sets the correct base path for the repository.

The frontend needs to know the backend API URL. I added it as a GitHub Actions secret:

text

API_BASE_URL=<https://todoappwasm-api.onrender.com/>

This keeps the frontend connected to the live Render backend.

Section 8: Bug fixed during deployment

Show:

- Login.razor
- CreateTodo.razor
- EditTodo.razor
- Logout.razor
- ViewTodos.razor

Speaking notes:

One real deployment bug I fixed was GitHub Pages routing.

After login, the app originally navigated to:

text

/ViewTodos

On GitHub Pages, that tried to open:

text

<https://umarsalem.github.io/ViewTodos>

But the app is hosted under:

text

<https://umarsalem.github.io/ToDoAppWasm/>

So I changed the navigation to app-relative routes like:

text

ViewTodos

This fixed the GitHub Pages 404 after login.

Section 9: Final test

Show:

- Create a new user
- Login
- Create todo
- Edit todo
- Delete todo
- Open Swagger
- Test health endpoint

Speaking notes:

This final test proves that the frontend, backend, authentication, and database are working together.

The app is not only running locally. It is deployed publicly with a real frontend URL, backend URL, hosted database, and CI/CD workflow.

Closing script

That is the full deployment of my ToDoAppWasm project.

In this project, I practiced Blazor WebAssembly, ASP.NET Core Web API, JWT authentication, Entity Framework Core, Docker, GitHub Actions, GitHub Container Registry, Render, PostgreSQL, and GitHub Pages.

The most important part for me was not only writing code, but learning how to connect everything together and debug real deployment problems.

Thank you for watching.

YouTube upload checklist

Title:

text

Deploy a Full-Stack .NET 8 Blazor App to GitHub Pages and Render

Description:

text

In this video I show my full-stack TodoAppWasm portfolio project built with Blazor WebAssembly, ASP.NET Core Web API, Entity Framework Core, JWT authentication, PostgreSQL, Docker, GitHub Actions, Render, and GitHub Pages.

Live demo:

<https://umarsalem.github.io/TodoAppWasm/>

Backend health:

<https://todoappwasm-api.onrender.com/health>

Swagger:

<https://todoappwasm-api.onrender.com/swagger>

Chapters:

text

00:00 Introduction

01:30 Live demo

04:00 Architecture overview

06:00 Backend code

10:00 Database and EF Core

13:00 Docker and GitHub Actions

17:00 Render deployment

22:00 GitHub Pages deployment

26:00 Deployment bug fix

30:00 Final test

33:00 Summary

Tags:

text

.NET 8, Blazor WebAssembly, ASP.NET Core, Render, GitHub Pages, PostgreSQL, Docker, GitHub Actions, EF Core, JWT Authentication

Final reminder

Do not show secrets on screen:

- Render database password
- Full database connection string
- JWT secret key
- GitHub secrets
- Personal access tokens

If a secret appears in the recording, blur it before uploading or record that part again.